

Introduction

The existential tension at the heart of modern cryptography, and hence fundamental vulnerability of all modern cryptography relies upon conjectured or computational security—unlike information-theoretic security (which is mathematically unbreakable, like this One Cipher)—is built entirely on conditional hardness assumptions with zero provided miasmatical proof against unknown unknowns.

All existing classic and Post Quantum proposals are predicated upon the existence of certain problem being inherently hard and taking exponential time to solve. If it turns out that P = NP, the mathematical bedrock of modern cryptography instantly dissolves. One cannot assume that a shortcut does not exist just because we haven't found it yet. All existing cryptography relies on the existence of known knowns and has no defence against the ultimate "unknown unknows".

Computation security exists today because these exist no **Proof of Security** for any existing cryptographic algorithm/cipher/function. With the existence of this published Proof-of-Security all existing forms of classic and post-quantum computational security self-deprecate and become obsolete. Instead of relying on the *assumption* that an algorithm is safe because **no one has publicly broken it yet**, Shannons perfect security exists having absolute information theoretic certainty that it can never be broken by an adversary.

Post-quantum cryptography was created because we lack the existence of any proof against the likes of **Shor's or Grover's algorithms** which mathematically proved that a quantum computer can easily solve these complexity-based math problems.

One Cipher based solutions forms the basis for the only **quantum-proof permanence**. Because it provides perfect secrecy, neither Shor's algorithm, Grover's algorithm or any other future unknown attack vectors can extract information from the One Cipher ciphertext.

This proof of security was written by a human, any typos or errors are my gift to you.

The Core Problem: No Hidden Uncertainty

In all existing symmetric cryptography, they use short keys (like a 256-bit AES key) to securely encrypt much larger messages (like a 1 GB file). This is conjectured to be possible because they rely on computational security and the assumption that the **message space contains high min-entropy** greater than the key min-entropy, or that a low entropy initialization vector (IV, like 12 bytes)) introduces freshness. However, as Shannon proved, if the message space has **zero min-entropy**, there exist no randomness or uncertainty in the cyphertext output itself, and the ciphertext becomes predictable and hence guessable. When min-entropy hits **absolute zero**, the adversary knows the exact distribution (or the exact single message) ahead of time.

According to Shannon's theorem for perfect secrecy, to ensure that the ciphertext reveals absolutely zero information about the plaintext, the information theoretical entropy of the key space H(K) must be greater than or equal to the entropy of the message space H(M), be used only once and be equal in length to the message plaintext.

In *entropic security*[1], the cryptosystem needs to guarantee that the entropy of the message space is **high from the point of view of the adversary**. This paper uses Shannon entropy as the basis for **verifiable *unconditional information security* which remains secure** against adversaries with unlimited computational power, resources, memory space, and time.

$$\text{Key} \rightarrow \text{One Cipher} \rightarrow \text{Zero Entropy data} = \text{Key}$$

In cryptography, entropic security ensures that a cipher remains secure even if the attacker has partial information about the secret key, provided the key still contains enough underlying uncertainty (entropy). This means to ensure maxim entropy cipher output; the original key must have maxim entropy as no deterministic f(x) or cipher can add entropy.

E*ntropic security*[2] *defines and codifies via a provided test case (Entropic Security Check) Shannons original Information* Theoretic Security definition of a **random key**[3]**,** to ensure the **secure key** remains **unpredictable** and possesses **high min-entropy from the adversary's perspective**, even after observing the ciphertext, and when the

---

[1] Nobody knows what *entropic security* really is, so in any cryptographic based debate we will always have the advantage."

[2] **The Rule:** A cryptosystem has Shannon's perfect information secrecy if the ciphertext (C) and the plaintext (M) are statistically independent, additionally the key has high entropy/uncertainty. Learning the ciphertext gives an eavesdropper absolutely zero new information about the original message, or the key.

[3] "a sequence chosen from a set of equally likely possibilities (a uniform probability distribution) where the key entropy equals its maximum possible value"

plaintext message **may have zero entropy**. E*ntropic security does not remove or redefine any of Shannons original requirements within this* ***Security-Proof.***

In cryptography and information theory, the One Cipher processing an **entropic key** (a key with high uncertainty or randomness, contains zero collisions and passes a NextBitTest) with a **zero-entropy value** (a completely predictable constant, like all zeros) results in an **output that retains the exact same entropy as the original key**.

A perfect secure cryptographic system **cannot rely** on the secrecy or randomness of the message itself, as these are frequently predictable. Therefore, the **entire burden of security rests on the key**. High min-entropy key, ensures that even if an adversary knows exactly *what* you might be saying and captures the *ciphertext* of it, the key space remains so vast and uniform that calculating or guessing the key remains computationally impossible.

Entropic Security for OTP:

- **The Math:** Let K be your entropic key and Z be a string of all zeros ($000...0$).
- By definition of the XOR ($\oplus$) operation: $K \oplus Z = K$.
- Because the output is identical to K, no information is gained or lost, meaning the **Shannon entropy** $H(K \oplus Z) = H(K)$.

If the entropy drops or changes during a One Cipher encryption operation, it signals a flaw in the One Cipher implementation, memory management, or medley secure key generation.

The Information Theoretic Security-Proof, for the Medley-One Cipher is based upon information theory as defined via the following six principles:

1. Theoretical Proof...

From Shannons 1949 paper, Communication Theory of Secrecy Systems, that an encryption system has perfect secrecy:

An encryption system has **perfect secrecy** if the conditional probability of any plaintext message given a ciphertext is equal to the unconditional probability of that message (P(M|C) = P(M)), meaning the ciphertext yields no additional information about the underlying plaintext

Definition of Perfect Secrecy

- **Independent Distributions:** In Shannon's formulation from *"Communication Theory of Secrecy Systems"* (1949), a cryptosystem achieves perfect secrecy when the a posteriori probability of the message M given the ciphertext C is identical to the a priori probability of M
- **Mathematical Condition:** P (M = m | C = c) = P (M = m) for all messages m and ciphertexts c with non-zero probability. Equivalently, the mutual information between the plaintext and ciphertext is zero: I (M; C) = 0

Core Proof Insights & Key Theorem

1. **Equal Likely Plaintexts/Ciphertexts:** Shannon demonstrated that for a system to achieve perfect secrecy, observing the ciphertext E (K, M) = C must leave all possible plaintexts equally likely from the attacker's perspective, regardless of the attacker's computational power.
2. **The Size Bound (Entropy Condition):** Shannon proved that perfect secrecy requires the key space size (H(K), the entropy or uncertainty of the key) to be **at least as large** as the message space size (H(M), the entropy of the message):$H(K) \geq H(M)$.
3. **Implication for Key Length:** If messages are measured in bits and chosen from a space of n bits, the key must contain at **least n bits of truly random information**[4] ($|K| \geq |M|$).

[4] If the key entropy is lower than the message entropy, a theoretical adversary with unlimited computing power could eventually narrow down the possible plaintexts, meaning the system does not achieve perfect secrecy. **Claude Shannon mathematically proved that key entropy must be at least as great as message entropy** ($H(K) \geq H(M)$ by establishing that for an encryption system to achieve perfect secrecy, the ciphertext must reveal absolutely no information about the plaintext.

This mathematically validates that the One Cipher is a theoretically unbreakable cipher as it satisfies the core properties of the proof of security namely a variable uniform random key with entropy greater than the message, used only once, and key length ≥ information object.

The One Cipher makes the following Proof of Security claims:

a) $P(M = m \mid C = c) = P(M = m)$m) i.e. seeing a ciphertext doesn't give you any extra information about the plaintext. The probability of seeing a message m after the ciphertext has been observed is the same as the probability of the message without the ciphertext.

b) $P(C = c \mid M = m_0) = P(C = c \mid M = m_1)$) i.e. the probability of ciphertext c is equally likely for 2 different messages.

c) Key space size[5] (H(K), the entropy or uncertainty of the key) to be **at least as large** as the message space size (H(M), the entropy of the message):H(K) ≥H(M).

d) The Medley KG will always generate a keys:
Key Length: The key must be at least as long as the message (|K| ≥ |M|).
Uniform Distribution: Each key must be chosen uniformly at random from the key space (P(K=k) = 1/|K|).
Single Use: The key must be completely unique and used only once.

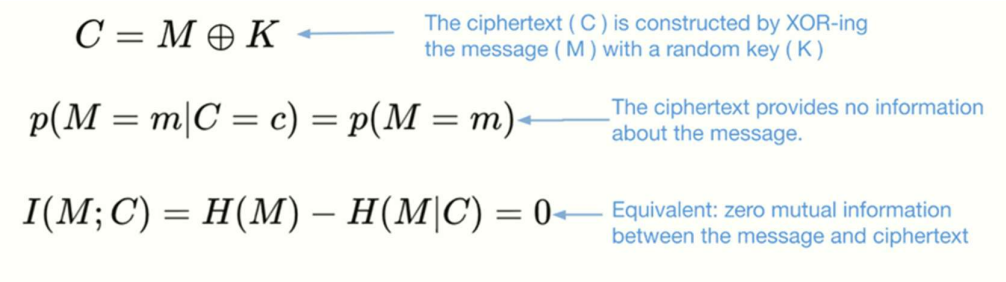


Figure 1 Summary of Claims

[5] The Key Generator key space has a minimum but not maximum ~ **34.36 billion** bits of entropy storage.

In Conclusion

The only mathematically unbreakable encryption scheme requires a key that is the exact same length as the message, used only once, and sampled from a perfectly uniform distribution. Such as the Medley One Cipher.

When Shannon entropy = Shannon min entropy, an attacker cannot predict the next outcome with any higher accuracy than a random guess.

Kerckhoffs's principle states that a cryptographic system must remain secure even if everything about it is public knowledge, except for the secret key. This means that the security of any information system is dependent on the security of the secret key.

No known function f(x) can increase the security or min-entropy or uniform distribution of the secret key. Claude Shannon mathematically reinforced this principle and introduced the concept of perfect secrecy using the XOR One-Time Pad (OTP).

According to Kerckhoffs's principle:

- An attacker knows the One Cipher (XOR algorithm) is being used.
- An attacker intercepts the Ciphertext (C).
- . Despite knowing the exact f(x) as the One Cipher (vernan cipher), the attacker cannot learn anything about the Plaintext (P) without the Key (K).

Knowing the One Cipher algorithm (Kerckhoffs's Principle) gives the attacker no advantage.

A bit stream with maximum Shannon entropy (ergo maximum information content) is nature aggregated random noise. It contains the maximum possible “information content” in terms of bits needed to represent it, it is computationally incompressible,

it also has no meaningful or useful information in the conventional sense.

Claude Shannon proved that achieving information-theoretic perfect secrecy demands that every key bit of a secure bit stream is chosen uniformly at random, meaning the min-entropy per bit is maximal.

The Medley Key Generator must be **self or independently verified** (using the supplied source code or AI prompts) to produce Shannon min-entropy output with zero collisions and Next Bit Test pass.

Proving:

- The knowledge of the Cyphertext is not helpful at all!
- The result is independent of the capability of any attacker or computational resources.
- The Medley One Cipher, based upon XOR as implemented, and has an output with proven and verifiable Shannon min-entropy = entropy, in accordance with above is provably secure against all attacks (known and unknown).

The above proof requires a "cryptographically secure" and verified Shannon min-entropy, bit stream (CSRBG) as the proof, in order to implement a truly secure one-time pad.

The Medley Key Generator (KG) is one such example of a Cryptographically Secure Non-Deterministic Random Bit Generators (CSNRBG).

It should be noted that a CSRBG cannot be computed Soley from a TRNG seed, as this would produce a CSPRNG key, which would not meet the requirements of the above proof.

2. Verifiable Proof…

Self-verifying cryptography is the safest mindset when dealing with digital security, never trust ‘so called experts or conflicted entities like NIST.

"Trust but verify. Or don't trust at all…."

In order to achieve Information Theoretic Security, the outputs from each cipher must be validated against the intersection of Shannon min-entropy and Shannon entropy to ensure a perfect uniform distribution.

Only after a Pass result (validation) can the specific instance of the cipher output be considered as Information Theoretic Secure.

No cipher can be generically considered secure unless each and every cipher output passes validation.

The following code is provided to allow a binary self-validation of any claimed proof of security against any cipher output data stream.

Shannon Entropy

```
public static (double ShannonEntropy, double MinEntropy) Calculate(byte[] data)
{
    if (data == null || data.Length == 0)
        return (0.0, 0.0);

    int[] counts = new int[256];
    foreach (byte b in data)
    {
        counts[b]++;
    }

    double shannonEntropy = 0.0;
    int maxCount = 0;
    double len = data.Length;

    foreach (int count in counts)
    {
        if (count > 0)
        {
            if (count > maxCount) maxCount = count;
            double p = count / len;
            // Shannon entropy summation: -p * log2(p)
            shannonEntropy -= p * Math.Log2(p);
        }
    }

    double minEntropy = -Math.Log2((double)maxCount / len);

    return (shannonEntropy, minEntropy);
}
```

// Test vectors, for range limits.

// Entropy = 8

```
byte[] allValues = Enumerable.Range(0, 256).Select(i => (byte)i).ToArray();
```

// Entropy = 0

```
{ "Test Zero Entropy Bytes", new byte[size]},
{ "Test Maxium Entropy Bytes", allValues},
```

// Birthday attack or collisions:

```
public static int CalcCollisions(byte[] data)
{
    // Group by value and filter groups with more than 1 element
    var collisions = data.GroupBy(x => x)
                         .Where(g => g.Count() > 1)
                         .Select(g => g.Key);
    return collisions.Count();
}
```

AI Versions

1. Prompt: "calculate Shannon entropy and min entropy in bits per 8 bits for this hex encoded byte array
2C9D8EFC726469F6974DF4EC05C9292360CDE3C66F5510175BF7C13E532481A8

2. Prompt: “verify maximum number of collisions is zero for this hex encoded byte array
2C9D8EFC726469F6974DF4EC05C9292360CDE3C66F5510175BF7C13E532481A8

3. To test any TRNG/QRND, classic AES/BATTON or PQC use prompt: calculate Shannon min entropy in bits per 8 bits for this hex encoded byte array <add hex encoded output stream>

Example Results

Range Tests

Test Zero Entropy Bytes: Shannon-entropy 0.0000 min-entropy 0.0000
Test Maximum Entropy Bytes: Shannon-entropy 8.0000 min-entropy 8.0000

Entropy Tests

Medley One Cipher Bytes: Shannon-entropy 5.0000 min-entropy 5.0000
AESGCM Cipher Text Bytes: Shannon-entropy 4.8125 min-entropy 4.0000
Latice Ring-LWE Cipher Text Bytes: Shannon-entropy 4.9375 min-entropy 4.0000

Collision Tests

AESGCM Collisions 3.0000
Medley One Cipher Collisions 0.0000II

Entropic Security Check

In cryptography and information theory, **entropic security** dictates how leakage behaves when high-entropy data undergoes transformations. When you perform a bitwise One Cipher **transformation** between a highly random variable (the entropic key) and a completely deterministic, invariant block (zero entropy, such as a buffer populated entirely with 0x00 or a single constant string), the resulting output's Shannon entropy **must exactly match** the initial key's entropy. This happens because One Cipher encryption with a static bias shifts the distribution deterministically **without changing** its underlying probability vectors or structure. Because the One Cipher transformation is a **one-to-one (bijective) mapping**, it preserves the structural properties of the original key and hence entropy of the output data.
To achieve true cryptographic uncertainty of the output distribution, **the initial key must be a verifiable uniform probability vector**.

```
byte[] key = RandomNumberGenerator.GetBytes(size);
byte[] zeroEntropyBuffer = EntropyCalculator.ZeroEntropyByteArray(key.Length);
byte[] xorResult = OneCipher.Encrypt(key, zeroEntropyBuffer).Skip(16).Take(key.Length).ToArray(); // remove One Cipher 16 byte authentication tag
bool p = EntropyCalculator.EntropicCheck(key, xorResult);
Debug.WriteLine($"OneCipher->Entropic Security Pass: {(p ? "Yes" : "No")}");
```

Figure 2 One Cipher Entropic Security Test Case

All existing classic and PQ ciphers change the underlying probability vectors or structure of the message and hence cause an entropy difference between the key and the output ciphertext.

```
// AES
byte[] nonce = new byte[12];
byte[] tag = new byte[16];
byte[] ciphertext = new byte[key.Length];
using (var aesGcm = new AesGcm(key, 16))
{
    aesGcm.Encrypt(nonce, zeroEntropyBuffer, ciphertext, tag);
}
p = EntropyCalculator.EntropicCheck(key, ciphertext);
Debug.WriteLine($"AES->Entropic Security Pass: {(p ? "Yes" : "No")}");
```

Figure 3 AES Entropic Security Test Case

Entropic Security Test Results

One Cipher->Entropic Security Pass: **Yes**
AES->Entropic Security Pass: **No**

Attribute	Status	Description
Probability Vectors	Unchanged	The relative frequencies (probabilities) of the elements remain exactly the same; they are just mapped to new positions.
Entropy (Uncertainty)	Unchanged	No information is gained or lost. The amount of randomness or predictability in the dataset remains identical.
Data Structure	Unchanged	Relationships, patterns, and distances between data points are completely preserved.

Figure 4 One Cipher, structural Impact of Static Bias

Security Conjecture

If any cipher claims to be secure, the tested output using the above functions, shall a) pass the Entropic Check, b) have measured Shannon min-entropy within 90% of Shannon entropy to be considered uniform and hence unguessable. Scores which deviate by more than 90% may reveal hidden patterns, biases, or repeating sequences that algorithms or bad actors can exploit. The 10% variation was selected based upon the table below:

Impact of a 10% Discrepancy on Bits of Guessability

To see how a 10% divergence affects the number of bits of guessability, consider a cryptographic key or password generated with a non-uniform distribution:

Metric	Uniform Distribution (Baseline)	Non-Uniform Distribution (10% Divergence)
Shannon Entropy (H)	**10 bits**	**10 bits** (High average randomness)
Min-Entropy (H_∞)	**10 bits**	**9 bits** (10% lower than Shannon)
Max Probability (p_{max})	$2^{-10} = \frac{1}{1024} \approx 0.098\%$	$2^{-9} = \frac{1}{512} \approx 0.195\%$
First-Guess Success Rate	1 in 1,024 tries	**1 in 512 tries** (Twice as easy to guess)

Figure 5 In comparison Grover's algorithm reduces the effective security strength (or work required to brute-force) of a symmetric cryptographic key by the same 50% (halving the bit security level) ...

AES as tested, has a 50% to 60% a deviation in values today well in excess of 10% defined herein which demonstrates the proven weakness or deviation from uniform output of the AES key schedule and algorithm design.

3. Additional considerations.

a) Malleability and active attack vectors

Yet another issue has to be considered, namely the message authentication, which is not usually implemented in Vernam Ciphers.

Malleability is common to all stream ciphers (Hanaoka, 2008). Also, the XOR operator is vulnerable to a known-plaintext attack, since XOR (plaintext, ciphertext) = key. Active attackers can also modify messages in transit; a weakness shared with all stream ciphers.

The Medley solution adds proprietary authentication to each key to addresses these well-known issues.

For clarity, there are no Public Key functions involved within any InfoSec or One Cipher use cases.

b) Confused Deputy

Ambient authority attacks, where permissions are attached to the execution context overall identity rather than bound explicitly to each specific request or operation/encryption context. The One Cipher algorithm is an Authenticated Encryption with Associated Data (AEAD) algorithm and hence ties the AAD to the ciphertext. If an attacker intercepts a valid encrypted blob and tries to trick a service (the deputy) into decrypting it using a different user's context or destination, the decryption operation fails before any decryption operation is processed.

c) Malleable ciphers

When a cryptographic system is **malleable** (like the One Cipher or vernan class of ciphers or stream ciphers without a MAC), an attacker can alter the encrypted data. If an attacker knows the structure of the underlying plaintext, they can flip bits to change critical values—such as a user ID from 1002 to 0001 (admin). When the application decrypts this tampered ciphertext, it trusts the result completely. The application (the **deputy**) becomes **confuses. Standard authenticated encryption (like Encrypt-then-TAG) protects the ciphertext itself from malleability. However, it doesn't protect the context in which that ciphertext is used.** This is where Additional Authenticated Data (AAD) is used in the One Cipher.

d) Side channel/power analysis

The One Cipher or bitwise XOR operation is mathematically perfectly balanced (0 and 1 outputs are equally likely given a random key) and executes in **constant time** on standard processors or physical logic gate implementations. But implementation Hamming Weight/Distance Leakage, logic gate transition period glitches, CPU pipeline and cache-timing side channels and other specific implementation details still require careful consideration.

Factor	Mathematical XOR Cipher	Physical XOR Implementation
Execution Time	Constant (1 cycle)	Constant, but execution pipelines can leak.
Power Consumption	Theoretical zero-variance	High variance due to CMOS switching and glitches.
Data Transmission	Isolated bitwise logic	Leaks via EM radiation and power line fluctuations.

Figure 6 Summary of Vulnerability vs. Mitigation guide

To safely implement the One Cipher in **high-security hardware**, engineers must use physical countermeasures such as **glitch-free dual-rail logic**, **cryptographic masking/blinding** (splitting the secret key into random shares), or **dummy power clamps** to hide the real signal.

Software implementations of the One Cipher make use of specific compiler directives and execute a **masked XOR operation** to prevent power profiles from correlating to the secret key.

e) Lastly Keys

Kerckhoffs's principle, holds that a cryptosystem should be secure, even if everything about the system, except the key, is public knowledge.

Thus, cryptographic keys should generally be generated secretly and uniformly at random in the cryptosystem's key domain (universals); that is in the set of valid keys for the cryptosystem. What makes a key valid depends on the cryptosystem and often parameters (typically including key size).

Since the adversary may know how our system works (Kerckhoffs's principle), we need to find a way to generate keys that don't only depend on a deterministic computation (which the adversary could reproduce). Therefore, all keys must depend on something that the adversary does not know. Such a thing is called a “true” random value.

The proprietary KMS-Medley Key generator (KG) takes a QRNG (TRNG) seed and produces a Cryptographically Secure (non-deterministic) Random Bit Generators (CSNBRNG) in accordance with Shannon min-entropy. Meaning each bit in the key sequence is equally likely to occur.

Medley is based upon a proprietary KG, which generates a CSNBRNG key stream upon demand, the KG TRNG seed is never stored and has a secure ephemeral (one time) existence.

Medley supports user defined KG-universals to enable the creation of arbitrary key domains to suit a wide range of operational deployments of an ITS-KMS.

4. Security Principles

SCI/Codeword or Commercial-Need to Know

The solution supports Top Secret/Sensitive Compartmented Information or commercial need-to-know, cloud-based solutions (hardware or software backed).

The ITS-KMS supports the implementation of Medley 'Universals" which guarantee separation between cryptographic Key Generators, hence ensuring ITS based separation of entangled Key Generators.

All SCI compartments are cryptographically (ITS One Cipher) isolated from all other compartments.

To ensure backward compatibility all existing human based codeworks can be utilised to enforce "need to know" requirements for both military and commercial use cases.

5. Cryptographically Secure Keys

Riddle me this?
Can any deterministic function create entropy?
The answer is always no, it cannot...

It is a simple reality, that no computational function can create entropy as it is deterministic or a just a variant of a PRNG source of fixed entropy. A Key Generator is nondeterministic; hence it can create entropy via external real-world entropy. Thus, a Key Generator creates CSNBRNG keys. But the entropy is still limited to the entropy of the real-world source.

Information Theoretic security requires and is underpinned by the existence of a Cryptographic Secure Non-Deterministic Random Bit Stream generator anchored in a real-world entropy.

A Cryptographic Secure Non-Deterministic Random Bit Stream (CSNBRNG) variable length key is not a key or even a QRND number in the typical sense, it must be a cryptographically secure key stream QRNG, which is not derived, via any computational

algorithm. The key stream has to be derived (extracted) from a physical process (in the case for medley from a QRND source).

One cannot compute an algorithm to generate a 'key stream' as the result would be a deterministic and hence represent a predictable PRNG, not a cryptography secure key stream and be a non-deterministic key stream not computed directly from a seed/key.

6. Last Mile Mandatory Security Policy

1. All comminutions Shall be One Cipher (ITS) secured independent of communication media or sChannels security.

2. All InfoSec devices require mandatory biometric authentication, such as Windows Hello fingerprints or Android device fingerprints for InfoSec use cases.

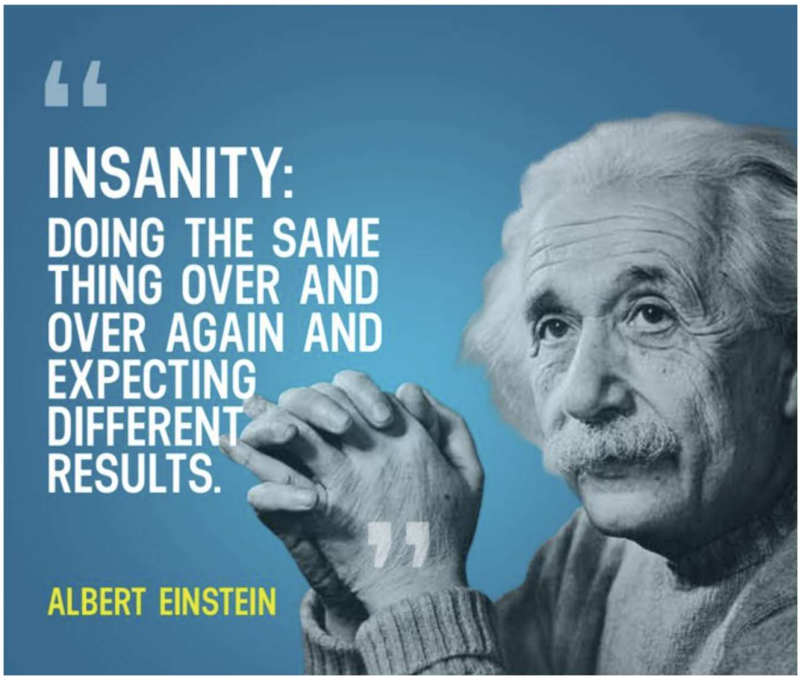
3. All Vaulted Objects shall be end-to-end Once Cipher (device to device) secured before the application of device to host and host One Cipher wrapped.

Conclusion

The Medley (ITS-KMS) creates a pairwise, potentially infinite length, identical secure (verified) bit stream (all bits of the key have equal probability) between two secure Identites (end point devices) according to this proof.



Insanity of Security without a ‘Proof of Security’



The quote "Insanity is doing the same thing over and over again and expecting different results" is often attributed to Albert Einstein.

The essence of the quote is to encourage people to be open to change, to learn from past experiences, and to avoid getting stuck in unproductive patterns…

Within cryptography this same insanity of developing algorithms and cipher which come with zero security i.e. doing the same thing over and over again and expecting different results" has become accepted as the norm.

Always Self Verify all cryptographic algorithms and ciphers **before use,** any other form of due diligence is nothing more than pixy dust or magic…

Self-verifying cryptography is the safest mindset when dealing with digital security, never trust ‘so called experts’ when they come with “zero proof of security” …

Secure keys?

Grover's is often described as looking for "a needle in a haystack" using quantum mechanics – it finds the needle in roughly the square root of the haystack size, which is vastly faster than checking each piece of hay one by one.

Complexity based algorithms which have no proof-of-security rely on problems that are intractable to brute-force search. Week or poor-quality keys which do not have Shannon entropy equal to min entropy will typically reduce the key search space by a similar amount as Grover's algorithm and does not require the existence of quantum computer.

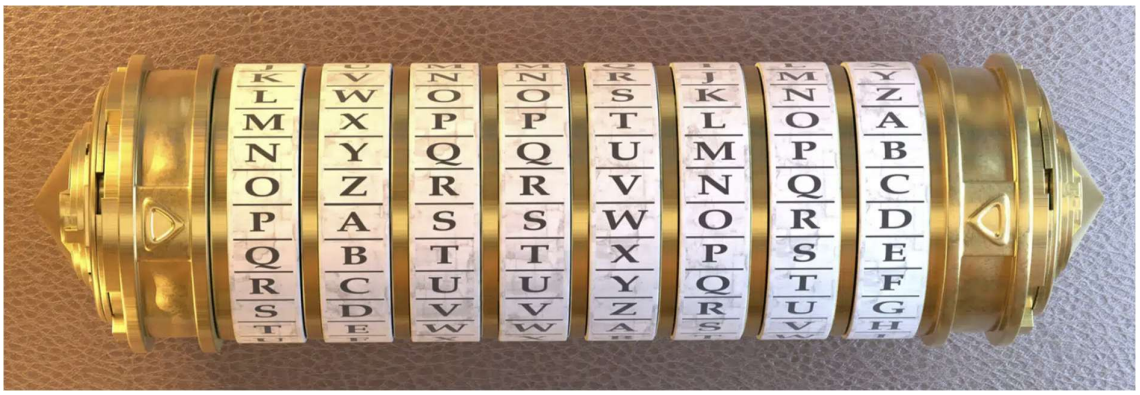


Figure 7 Security is always about entropy, not randomness or key length.

Shannon Perfect Security-> The One Cipher



Measured vs Actual Shannon min-entropy

According to the **Laws of Information Theory**, you cannot create *true* entropy out of nothing. Hence no deterministic functions f(x) cannot create or increase entropy, at best that can maintain the input value or decrease it. This proof contains code and AI prompts to measure both Shannon entropy and min-entropy, these need to be used with the following understanding.

Measured entropy can make poorly distributed data *look* perfectly random to statistical tests, the actual cryptographic security (min-entropy) is strictly limited by the uncertainty of the original input. To understand this phenomenon, we must separate how statistical tools measure data from how information theory defines its actual security.
The SOneCipher.GenerateKeyStream output bits are cryptographically scrambled, they possess a near-perfect 50/50 balance of 1s and 0s with no detectable patterns. If you run a Shannon entropy test or a statistical min-entropy estimator on this processed output, the test will be fooled into reporting **maximal entropy** (trending to 8 bits of entropy per output byte).
But actual min-entropy depends entirely on the size of the state space an attacker has to search. Because SOneCipher.GenerateKeyStream function is entirely **deterministic**, they do not introduce new randomness. Only the Medley ->Key Generator, add external randomness to increase Shannon min-entropy.

SOneCipher.GenerateKeyStream function is an excellent **entropy conditioner** (or extractor). It takes poorly distributed, biased real-world entropy and concentrates it into a close to uniform, compact key stream. However, it cannot act as an **entropy generator**. Once you stretch the processed output beyond the bit-strength of the unique input entropy, you are simply stretching a mathematical illusion of entropy or randomness.

The SOneCipher.GenerateKeyStream function hasn't violated information theory by creating new randomness; it has simply taken the original bits of uncertainty and **projected it across a massive computational space**, making it impossible for an observer to predict the next bit without knowing the internal state. It is computationally impossible to figure out what the next bit will be—unless they possess the exact internal state (the seed and current variables) of the generator, successfully "fooling" **Shannon entropy tests** and other statistical suites.

Example

You generate a 128-bit key, but hardware source only chooses from **16 possible variations**. The actual min-entropy of this input is only **4 bits** ($2^4 = 16$).

- You feed this input into the SOneCipher.GenerateKeyStream function and "squeeze" out a 256-bit output.
- The 256-bit output looks completely random. A statistical tool will measure its Shannon entropy and declare it has 256 bits of entropy.
- However, because the function is deterministic, those 16 unique inputs can only ever map to **16 unique outputs**.
- An attacker who knows your system architecture does not have to guess a random 256-bit number. They only have to run the 16 possible inputs through the function to see which output matches yours. Your system still only possesses **4 bits of actual min-entropy.**

Medley Test Results (256-bit keys)

The Medley Key Generator conducts real-time QA on 100% of the Secure Keys used within the One Cipher solution.

```
Debug Trace:
Medley SecureRandom->NextBitTest Pass: Yes
Medley SecureRandom->Has Collisions: No
Medley SecureRandom->Shannon-entropy 5.0000 min-entropy 5.0000
Medley SecureRandom->Miller Madow Entropy 5.4844
```

Acknowledgment

As a general statement: "Claude Shannon's concept of perfect secrecy (or information-theoretic security) is mathematically achievable within the One Cipher as defined via its proof of Security, but system-level perfection does not exist in practice."

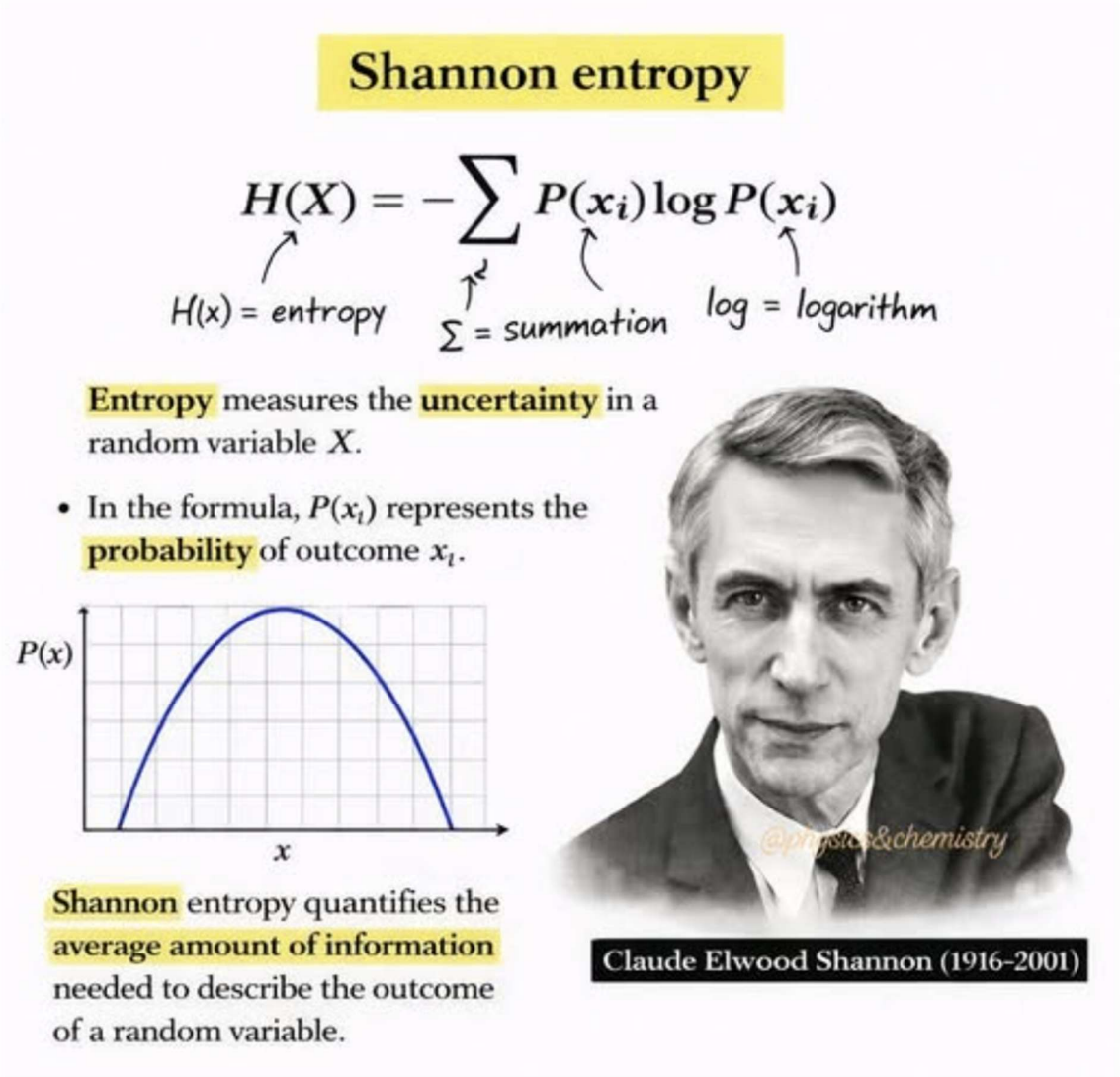


Figure 8 The real hero behind this proof, without who, ***verifiable entropic security*** *would be impossible today.*

Taxonomy of Shannon Abuse

This section **is not part of the formal Proof of Security** but rather provided to allow readers to spot Shannon abuse and misuse by vendors…

The Shannon Hustle: How Vendors Abuse Perfect Secrecy to Sell You Less Than You Think…

A vendor presents an encryption product. Somewhere between slides seven and twelve, the word “Shannon” appears. The claim takes different forms: “achieves Shannon Perfect Secrecy,” “extends Shannon’s theorem to reusable keys,” “provides information-theoretic security equivalent to the one-time pad”; but the structure is identical every time. The product uses keys far shorter than the messages it encrypts. Shannon’s 1949 theorem says that is impossible under his definition of perfect secrecy. One of these two things is wrong.

If one examines the mathematics behind such claims. In every case, the answer is the same: the vendor has either quietly changed Shannon’s definition, confused the size of the key *space* with the entropy Shannon’s theorem requires per encryption operation, or proved a security property in a restricted formal model and marketed it as if it holds in general deployment. The resulting product may or may not even provide good computational security. But it does not provide what the word “Shannon” is meant to imply, and the gap between the implication and the reality is **where the hustle lives.**

Vendor marketing invokes Shannon’s name for schemes that do not meet Shannon’s conditions. Each pattern employs a different mechanism to bridge the gap between the claim and the often-missing maths, in every case, they come with **zero proof of security**.

Factoid: The entire edifice of modern cryptography operates with short keys. AES-256 encrypts arbitrary-length messages with a 256-bit key. ML-KEM-768 establishes a shared secret with a combined public key and ciphertext of a few kilobytes. None of these schemes claim information-theoretic security. All of them claim ***computational* security**: breaking the scheme requires solving a mathematical problem believed to be intractable for any adversary operating within the bounds of computing power as currently understood.

It is impossible to mathematically prove that a state actor or a brilliant mathematician hasn't secretly found a shortcut to break the **Advanced Encryption Standard (AES/BATON)** and chosen to keep it a closely guarded secret. As one cannot definitively prove a negative.

In the world of intelligence and cyber warfare, a broken cipher is worth far more as a silent listening tool than as a public announcement. The decision to keep a broken cipher or a software vulnerability secret is driven by the fact that information control yields strategic leverage, whereas public disclosure destroys that leverage.

Historical and Modern Analogies

Era / Conflict	The Asset	The Strategic Decision
World War II	The German **Enigma Machine**	The Allies kept the decryption of Enigma (Ultra) a absolute secret. They even allowed certain smaller attacks to happen to avoid tipping off the Germans that the code was broken.
Cold War	**Project VENONA**	The US Army's Signal Intelligence Service decrypted Soviet messages for years. The program remained classified for decades to prevent the USSR from realizing their codes were compromised.
Modern Cyber Warfare	**Zero-Day Exploits** & State-backed Access	Agencies like the NSA (US) or Unit 8200 (Israel) maintain classified repositories of vulnerabilities to maintain access to target networks rather than reporting them to vendors.

"The cryptographic establishment is hostile to new ideas."

The Qualifier Insertion

The most common pattern is to add a qualifying word before "perfect secrecy" and hope the reader does not notice that the qualifier changes the definition. "Operational perfect secrecy." "Extended Shannon secrecy." "Practical perfect secrecy." "Quantum perfect secrecy." Each of these compounds borrows Shannon's authority while weakening his guarantee.

These and other like terms does not appear in Shannon's paper or in any subsequent textbook on information theory that I have been able to find. It has been coined by vendors or vendor-affiliated researchers to describe schemes that achieve *something like* perfect secrecy under *certain operational conditions*: typically, conditions that restrict the adversary to polynomial-time computation, or that assume the adversary cannot observe certain side channels, or that bound the number of messages encrypted under one key.

Within the proof, we often use the qualification "Perfect Information Security", rather than Perfect Security, this is to avoid the obvious misunderstanding of the most board sense of security and restrict the proof to be solely applicable to 'digital information' or InfoSec domain.

The Entropy Confusion

This is the most technically interesting pattern and the one most likely to confuse a sophisticated reader. It works by conflating the *entropy of the key selection space* with the *entropy Shannon's theorem requires per encryption operation*.

The entropy of the selection space is a real and meaningful quantity. But it answers a different question than the one Shannon's theorem asks. Shannon requires that the key entropy *per encryption operation* be at **least as great** as the message entropy. If the selected pad is applied to a 1-megabyte message, the question is whether the pad carries 8 million bits of independent entropy applied to those 8 million bits of message, not whether the pad was chosen from a space with 27,000 bits of combinatorial entropy. The pad might provide excellent computational security (an adversary trying to guess which permutation matrices were selected faces an enormous search space), but the information-theoretic guarantee is bounded by the operational key entropy, not the selection entropy.

The confusion is often genuine rather than deliberate. Researchers working in combinatorics or quantum information theory may use "Shannon entropy" in its information-theoretic sense, the entropy of a random variable; and apply it correctly to the random variable representing the key selection. The marketing team then interprets "Shannon entropy" as a reference to Shannon's secrecy theorem, and the claim migrates from a correct statement about combinatorial complexity to an incorrect statement about information-theoretic security.

Domain Switch

A third pattern proves a legitimate security property in one formal model and then markets the result as if it holds in the model where customers actually deploy encryption. The most prominent examples involve quantum ciphertext.

Several published papers prove that if the ciphertext is transmitted as a *quantum state* rather than a classical bitstring, certain security properties become achievable with shorter keys. This is correct. Shannon's theorem is a theorem about classical communication: it assumes the adversary receives a classical ciphertext and can store and analyse it indefinitely. Quantum ciphertexts behave differently: the no-cloning theorem prevents the adversary from copying them, measurement disturbs them, and the adversary must choose what to measure without the option of trying again. These physical constraints change the information-theoretic calculus.

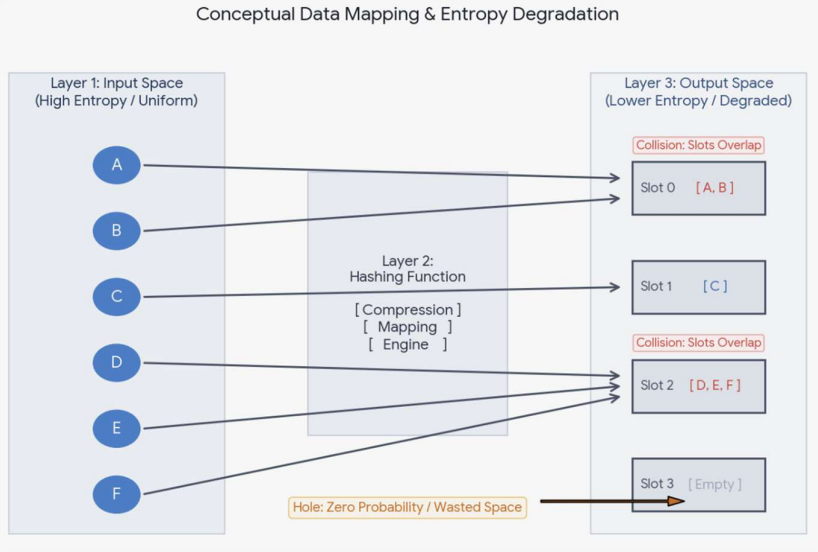
Every real-world deployment I have seen described in vendor literature transmits ciphertexts as classical bits over classical networks. In that setting, Shannon's classical theorem applies in full force, and the quantum ciphertext results do not rescue the claim. The security property proved in the quantum model does not transfer to the

classical deployment model. The vendor is citing a theorem that applies in a world different from the one the customer inhabits.

Hash /Key Derivation Functions increase entropy Claims

A hash function or Key Derivation function maps an arbitrarily large input space into a fixed-size output space. Because the input space is typically larger than the output space, the Pigeonhole Principle dictates that collisions are mathematically inevitable.

When treating a hash function as a random mapping, some output slots receive multiple inputs (**collisions**), which mathematically leaves other output slots empty (**holes**). This distribution asymmetry inherently results in an **entropy loss.**



Due to the Pigeonhole Principle, this compression inevitably results in non-uniform mapping, leading to collisions (multiple inputs sharing a single slot) and holes (empty, unmapped slots). As a result, the uniform probability distribution of the input is compressed unevenly, resulting in a **degradation of information entropy**.

In the fact of proven inability to increase entropy, they mostly claim to expand **effective security strength, increasing work factor** and **conditioning** distributed randomness (what ever this means for cryptographic security).

Example
Imagine you create a 6-character password (very low security strength). A cryptographic system can take that weak password and run it through a Key Derivation Function (KDF). This function transforms your short password into a 256-bit cryptographic output, but with the same uncertainty/randomness/entropy as the original 6-character password.

A fundamental rule of information theory is that **you cannot create entropy (uncertainty) out of thin air.**

- A standard 6-character alphanumeric password has an entropy of roughly **30 to 40 bits** depending on the character set used.
- Running it through a KDF outputs a **256-bit key**, but that key only has the *effective strength* of the original 30–40 bits of entropy.
- If an attacker knows the exact 256-bit key space represents a 6-character password, they only have to guess among the original 6-character combinations, not the KDF output of 2^256 possibilities.

So, why do NIST and others claim it increases uncertainty?
It once again comes down to so called **computational complexity** and **structural obscurity**.

Overcomes Shannon" Claim

Some papers and vendor materials go further than the domain switch and directly claim to have "overcome," "extended," or "gone beyond" Shannon's theorem. I flag this as a distinct pattern because the framing implies that Shannon was wrong, or that his result has been superseded, rather than that the scheme operates outside his theorem's scope.

Shannon's theorem has not been overturned. It remains true in the classical model under the original definition, and it has been true for 77 years without a single counterexample. Results that achieve shorter keys do so by changing the model (quantum ciphertext), changing the definition of security (entropic security, discussed below), or changing the adversary's capabilities (computational bounds). Each of these is a valid research direction. None of them "overcomes" Shannon, any more than building an airplane "overcomes" gravity. The airplane operates by engaging with different physical principles (lift, thrust); gravity is still there. Shannon's theorem is still there. The vendor who tells you they have overcome it is either confused about the scope of their own result or hoping you will be.

The Entropic Security Bait-and-Switch

The core idea: if the plaintext distribution has sufficiently high min-entropy from the adversary's perspective, then an encryption scheme with keys *shorter* than the message can still provide a meaningful information-theoretic security guarantee — not perfect secrecy, but a guarantee that the ciphertext does not help the adversary predict any function of the plaintext.

The bait-and-switch works by marketing entropic security without the min-entropy caveat. A vendor describes their scheme as "information-theoretically secure with short keys" and cites the academic literature correctly. The missing sentence is: "…provided the plaintext has min-entropy close to its bit length." That sentence eliminates most enterprise use cases. Encrypted database fields, log entries, API payloads, email bodies, and document files all have extensive structure that reduces their min-entropy far below their bit length. For these data types, the entropic security[6] guarantee weakens proportionally, and the scheme falls back to relying on whatever computational hardness assumption underlies its key expansion or permutation mechanism.

Within this paper the context of entropic security and information theory, means if one takes a cryptographically secure **entropic key** and encrypts/transforms it with a **zero-entropy string** (a completely predictable string, like all zeros), **the resulting output** retains the **exact entropy of the original key**. Using a zero-entropy input is a standard baseline to verify that your key generation, cipher or function f(x) aren't artificially degrading or inflating the entropy pool.

[6] Entropic security was first formally defined in cryptography by Alexander Russell and Hong Wang in their foundational paper *"Entropic Security and the Encryption of High Entropy Messages"* (Eurocrypt 2005). Unlike a traditional One-Time Pad (which requires a perfectly uniform key as long as the message), an *entropically secure encryption (ESE)* scheme allows you to use a **much shorter key** ($k \approx n - t$). This paper codified Entropic security, to be a codification of Shannon defined **random keys** which is additional to Shannon proof, not a reduction in security.

Computational Entropy Claims

Computational entropy—is a vendor idea that while cryptographic functions/ciphers/algorithms might be proven mathematically deterministic, their functions outputs, from a human perspective, look indistinguishable from true randomness to an adversary with finite computing power and time.

Pseudo-entropy: A system possesses computational pseudo-entropy if it looks statistically indistinguishable from a truly uniform, high-entropy distribution to any observer constrained by polynomial time. Even if a 256-bit key was deterministically produced from a 128-bit seed, a computationally bounded attacker cannot tell the difference—meaning it effectively behaves with 256 bits of entropy under **contrived and often NIST standardised** complexity constraints.

Fact: No deterministic or standard functional cryptographic operation can create or add entropy; it can only preserve, transform, or reduce it. Output entropy H(Y) is always less than or equal to input entropy H(X) for any deterministic f(x): knowing X leaves zero uncertainty or entropy about Y.

Breaking down the cases:

1. **If $f(x)$ is a bijection (1-to-1 mapping):** No data is lost, so the entropy remains exactly the same: $H(Y) = H(X)$.
2. **If $f(x)$ is not a bijection (e.g., a hash function like SHA-256):** Collisions can occur, meaning multiple inputs map to the same output. This discards information, so entropy decreases: $H(Y) < H(X)$.

Therefore, **no cryptographic function can ever add raw information-theoretic entropy.**

Why It Feels Like Entropy is Added when it is not:

- **Expansion / Whitening**: Ciphers or hash functions map low-entropy or structured input (like a password or text) to a **uniform-looking** output bitstring.
- **Diffusion and Confusion**: Shannon's properties make patterns **hard to see**. The output ***looks* random** and has high-per-bit entropy, but the *actual amount of uncertainty (min-entropy)* is constrained and can never exceed that of the input secret key or input seed.
- **KDFs (Key Derivation Functions)**: make brute-forcing slow. They do **not** add entropy bits; they add computational work/time per guess.

The Proof

Core Proof (Information-Theoretic)

- **Deterministic functions**: Standard cryptographic functions (hash functions, block ciphers, key derivation functions) are deterministic. For input X, output $Y = f(X)$.
- **Data processing inequality**: In information theory, processing a random variable through a deterministic (or randomized) function f cannot increase its mutual information or entropy. Formally, $H(f(X)) \leq H(X)$.
- **Injected keys/seeds**: If a function takes inputs from multiple sources—such as message M and key K (e.g., $Y = E_K(M)$)—the joint entropy is $H(M, K) = H(M) + H(K|M)$.
- **Independence/Processing limit**: Applying a function $f(M, K)$ results in $H(f(M, K)) \leq H(M, K) = H(M) + H(K|M)$ (assuming independent keys/messages, $H(M) + H(K)$). The output entropy is bounded by the total input entropy. It never exceeds the sum of the input entropies.

Entropy can only come from external to the function sources of true randomness.

The Theorem Test

If the scheme does not meet Shannon's three conditions, which specific published theorem provides the security guarantee, in which formal model, and with what assumptions about the adversary and the plaintext distribution?

This is the question that separates honest innovation from name-dropping. Legitimate schemes operating outside Shannon's conditions have formal security theorems with stated assumptions. Entropic security results require a min-entropy bound on the plaintext. Quantum ciphertext results require a quantum communication channel. Computational security results require a hardness assumption or reduction (e.g., the security of AES, the hardness of LWE).

Ask the vendor to identify the specific theorem, the specific paper, and the specific assumptions. If the answer is "our internal analysis shows…" or "the combinatorial complexity of our key space provides…", the scheme has not been proven secure by any supporting unit test cases, which can be executed anyone outside the organization that sells it.

NSA Classified or Trade Secret Claims

NSA classifies and keeps secret its Type I algorithms to **Prevent Cryptanalysis:** Keeping the inner workings of the algorithms hidden stops hostile intelligence agencies from studying the code to find mathematical weaknesses or build decryption tools as they do not have any proof of security or public assessment or verification test suites.

The One Cipher remains a Trade Secret[7] for commercial not security rationales as it comes with a proof of security. But all practical deployments come with attack vectors which exist outside of the proof of security and hence may not be fully addressed by this proof and its associated test cases. Hence keeping them, a Trade Secret limits their footprint and reduces **possible attack surface** and hence the chance of commercial or unauthorized leakage of unknown attack vectors to any operational deployments. This is an example of the well-known **security in depth** paradigm.

The One Cipher and Medley Key generator and KDF uses unpublished, commercial proprietary designs, where secrecy is an added layer of defence to exposure of internal state attack vectors.

Identical to NSA and all existing High-Assurance ciphers the Medley-One Cipher, the use of commercial classifications/trade secrets is the norm not an exception, when no classified or commercial classic ciphers come with a proof of security.

[7] It is our legal right to commercially protect the over 26 years of R&D which went into the development of the Medley (codeword) suite of Intellectual Property developments to date.

Why the greatest Risks are Off the Map of Vendor claims?

The real danger lives in the “unknown unknowns” — risks we haven’t even imagined. These aren’t just blind spots. They are gaps in our mental models, born not from negligence but from the inherent complexity, emergence, and drift of all digital systems.

To make these invisible risks actionable, we must first reframe how we categorize them — starting with the Knowledge–Risk Matrix in Cybersecurity, which maps the four domains where assumptions, awareness, and understanding converge… or fail.



When **stitched together by an adversary**, armed with **time, perhaps AI automation, and a deep understanding of how real-world environments drift**, they form the kill chain behind the next breach for all systems except for the **One Cipher-Perfect Security.**

The Six-Question Summary

For reference, the complete decision framework:

1. **Length Test:** Key consumed per operation ≥ plaintext? If not → **computational security.**
2. **Randomness Test:** Key from full-entropy source, not expanded from a short seed? **If not → computational security.**
3. **Reuse Test:** Each key used exactly once and destroyed? I**f not → computational security.**
4. **Theorem Test:** Which published theorem, in which model, with which assumptions, provides the guarantee? **If "internal or not-in public domain Proof-of -Security" → unverified**.
5. **Cryptanalysis Test:** Can everyone outside the vendor **self-verify** the security of the cipher, and are the results of said test public available? **If cannot be self-verified → untested.**
6. **Proof-of-Security:** Does the solution have a verifiable proof of security? I**f not → seek a solution with a Proof-of-Security.**

A vender proposal that answers "no" to any of the first three questions provides computational security. That is not a condemnation; it is a classification. Evaluate the cipher on its self-verified results and Proof of Security. A name borrowed from a Shannon's theorem is not evidence that the theorem applies.

References

E. Shannon, "Communication theory of secrecy systems," in The Bell System Technical Journal, vol. 28, no. 4, pp. 656-715, Oct. 1949.

E. Shannon, "A mathematical theory of communication," in The Bell System Technical Journal, vol. 27, no. 3, pp. 379-423, July 1948.

Notes

1. Medley is a pairwise, cooperative key generation scheme, the Key Generation Cipher is provably (above) immune from all forms of attack vectors, as the CSNRBG keys are verifiable cryptographically uniformly distributed and hence secure.

2. Medley One cipher, at an abstract level, is essential the XOR (vernan cipher) of two corporately generated Shannon min-entropy bit streams.

3. The Medley protocol produces a set of variable length Medley Vectors; these entangle the Key Generators to produce an identical CSNRBG bit stream (of potentially infinite length).

3. Quantium Key Distribution (QKD) self-deprecates as QKD, requires both participating parties need a) the physical devices, b) a connection between those that allows to run QKD, and c) still some authentication method which either requires a shared secret or some complexity-theoretically secure signature scheme. In short in QKD the two parties have to know each other before communicating and pre-stage shared data first. This defeats the purpose of QKD itself.

4. All modern block ciphers produce deterministic or PRNG key schedules. Such deterministic key schedule, also called a key-schedule algorithm (at best exist as a PRNG computed output), takes the master seed/key and generates a unique set of subkeys or internal state based on a fixed, mathematical process or computable algorithm. This ensures that the same seed/key will always produce the same output, which is a core principle of how most modern and insecure ciphers operate.

5. No f(x) or any computer is capable of producing entropy, as such any seed entropy cannot be increased due to any key schedule expansion.

6. The Medley KG and One Cipher calculates both Shannon entropy and min-entry in order to determine secure keys.

7. For small sample sizes, Shannon min-entropy calculations should be considered alongside, the corresponding Miller Madow Entropy correction. A **higher Miller-Madow entropy value** means that your dataset contains a higher degree of randomness, uncertainty, or diversity.